

PROJECT BRIEF · V1 · 20 AUGUST 2026

Resurface

It turns the stuff you save and never watch into something you can actually search.

BUILT BY	Gulfam Khan
PURPOSE	A public artifact that carries senior-shaped engineering signals
DURATION	10–14 weeks at 1–2 h weekdays plus weekends
RUNNING COST	\$0 idle · under \$5/month at hobby scale
STACK	TypeScript · Postgres (Neon) · Cloudflare Workers · Cloud Run · OpenTelemetry · Claude
REPO	<code>/Users/Me/resurface</code> — week 1 code already written

00 Contents

01 Why this exists	08 The five engineering features
02 What it is	09 Phases 1–10, in depth
03 Who it is for	10 Cost estimate
04 How it works — architecture	11 Timeline and effort
05 The three diagrams	12 What you will be able to claim
06 Every technology, and why	13 Risks and honest warnings
07 What was rejected, and why	

01 Why this exists

The problem is not that you lack skill. It is that you have nothing you can show.

Your real work sits in two places, and neither can be pointed at. **Mylo is private** — 200+ backend modules, six production databases, reconciliation work that recovered ₹19.4M in misbooked payments, and none of it can be shown to a stranger. The **job pipeline** is public code you own, but you have ruled it out as a showcase, correctly: telling an employer you automated applications across nine job boards signals mass-applying, whatever the engineering quality.

So application forms ask "link us to something you have built" and the honest answer is currently nothing. That is the gap this project closes — and only that. It is not a business, not a startup, not a portfolio piece to be abandoned at 60%.

YOUR OWN OBJECTION, AND THE ANSWER

You said: *"only calling LLM API and getting response and showing to user is same as creating CRUD."*

You were right, and that objection is the design constraint for this whole project.

The answer: the LLM call is the cheapest, easiest part. The engineering exists because the operation being wrapped is slow (minutes, not milliseconds), expensive (real money per call), non-deterministic (the same input can give different output), and fails halfway through. CRUD is never any of those things. If this project ever degenerates into paste → call → render, *it has failed* and should be abandoned.

Everything in the phase plan exists to keep that from happening. The queue, the leases, the cost cascade, the resumability, the evals — each one is a thing you can only build because the wrapped operation is hostile.

02 What it is

Save-it-and-actually-find-it.

You forward a link — a YouTube video, a podcast episode, an article, or an audio file. The system transcribes it, works out what is *actually inside it* (a recipe, a workout, a book recommendation, a place worth visiting), and stores that as structured, searchable data instead of another unread bookmark.

The one-sentence version for a recruiter who is not technical: **"It turns the stuff you save and never watch into something you can actually search."** That sentence is deliberately boring and deliberately understandable. The interesting part is underneath, and it only comes up when an engineer asks.

Deliberately out of scope

EXCLUDED	WHY
Instagram & TikTok	Terms-of-service hostile and actively blocked. Identical engineering, none of the legal cliff. This is a judgement call worth being able to explain in an interview.
Live progress streaming (SSE)	Dropped on purpose. The UX is poll-for-status — slightly duller to demo, materially simpler to build. Can be added later.
Hybrid search (BM25 + embeddings)	Deferred. Plain Postgres full-text is enough to prove the pipeline; vector search is a separate project's worth of learning.
Accounts, billing, teams	Not the point. A handful of real users on a shared link is the target, not a product launch.

THE FAILURE MODE TO WATCH FOR

The temptation will be to make the UI nice. Resist it until week 9. A beautiful interface over a queue that loses jobs is worth nothing in an interview; an ugly page over a queue that provably survives `kill -9` is worth a great deal. The demo is the trace and the chaos report, not the front end.

03 Who it is for

Three audiences, and they want completely different things from it.

AUDIENCE	WHAT THEY WANT	WHAT THEY WILL ACTUALLY LOOK AT
You the real primary user	To close the biggest gap in your CV: everything you have built so far is <i>cron-driven</i> . You have never built a real queue.	Nothing. You will be inside it. The value is the 60% of this that is new to you — leases, visibility timeouts, backpressure, fairness, OpenTelemetry, evaluating non-deterministic output.
A non-technical recruiter	One sentence they can repeat to a hiring manager, and evidence you finish things.	The live URL, whether it works when they click it, and the README's first paragraph. They will not read code.
An engineer or EM interviewing you	Evidence you have made real trade-offs and survived real failures.	The fair-queueing SQL, the incident write-ups, the eval suite, and how you answer " <i>how do you know the extraction is correct?</i> "

YOUR UNFAIR ADVANTAGE – LEAN ON THIS

You are the reconciliation person. You have spent months proving that two systems which disagree about money can be made to agree, and you found ₹19.4M of misbookings doing it. Point that instinct at LLM output and you get the question almost nobody building AI products in 2026 can answer:

"How do I know the extraction is right, without a human checking it?"

Cross-check every extracted item against the transcript it came from. Score confidence per field, not per document. Gate low-confidence fields instead of showing them. Feed user corrections back as regression cases. That is reconciliation applied to a probabilistic system, and it is a genuinely distinctive thing to have built.

04 How it works — architecture

Two runtimes, one database acting as the message broker.

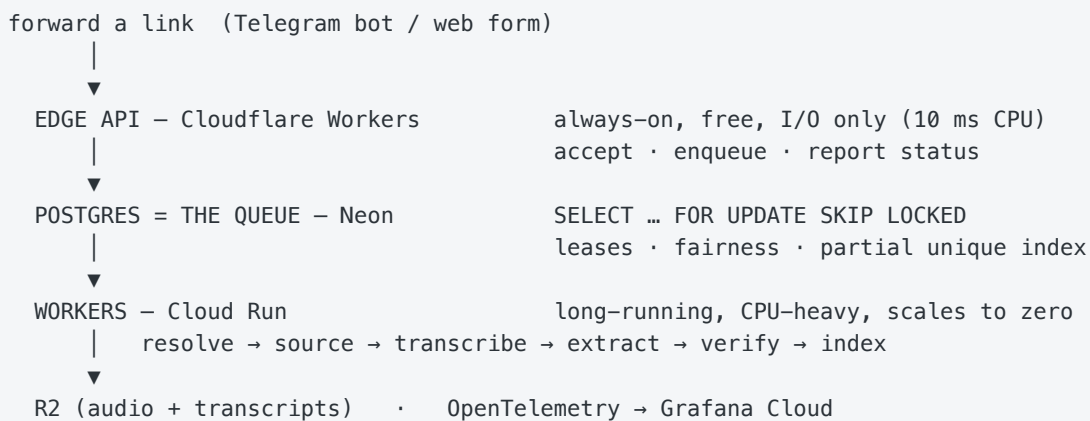
The shape is dictated by one hard constraint and one deliberate choice.

The constraint: the free front door cannot do work

Cloudflare Workers is free, genuinely never sleeps, and handles 100,000 requests a day — but it allows **10 milliseconds of CPU per invocation**. Waiting on the network does not count against it, so it can accept a request, write a row, and reply. It physically cannot transcribe audio. That forces a split between an always-on front door and an expensive back room, which is a real architectural pattern rather than an artifact of the free tier.

The choice: Postgres is the queue

No SQS, no Redis, no Kafka. `SELECT ... FOR UPDATE SKIP LOCKED` with leases, visibility timeouts and per-user fairness, implemented by hand. A managed queue would hide every one of those mechanisms behind a configuration flag — and those mechanisms *are the curriculum*. You cannot claim to understand backpressure because you set `maxReceiveCount` in a console.



05 The three diagrams

One for the shape, one for the lifecycle, one for the money.

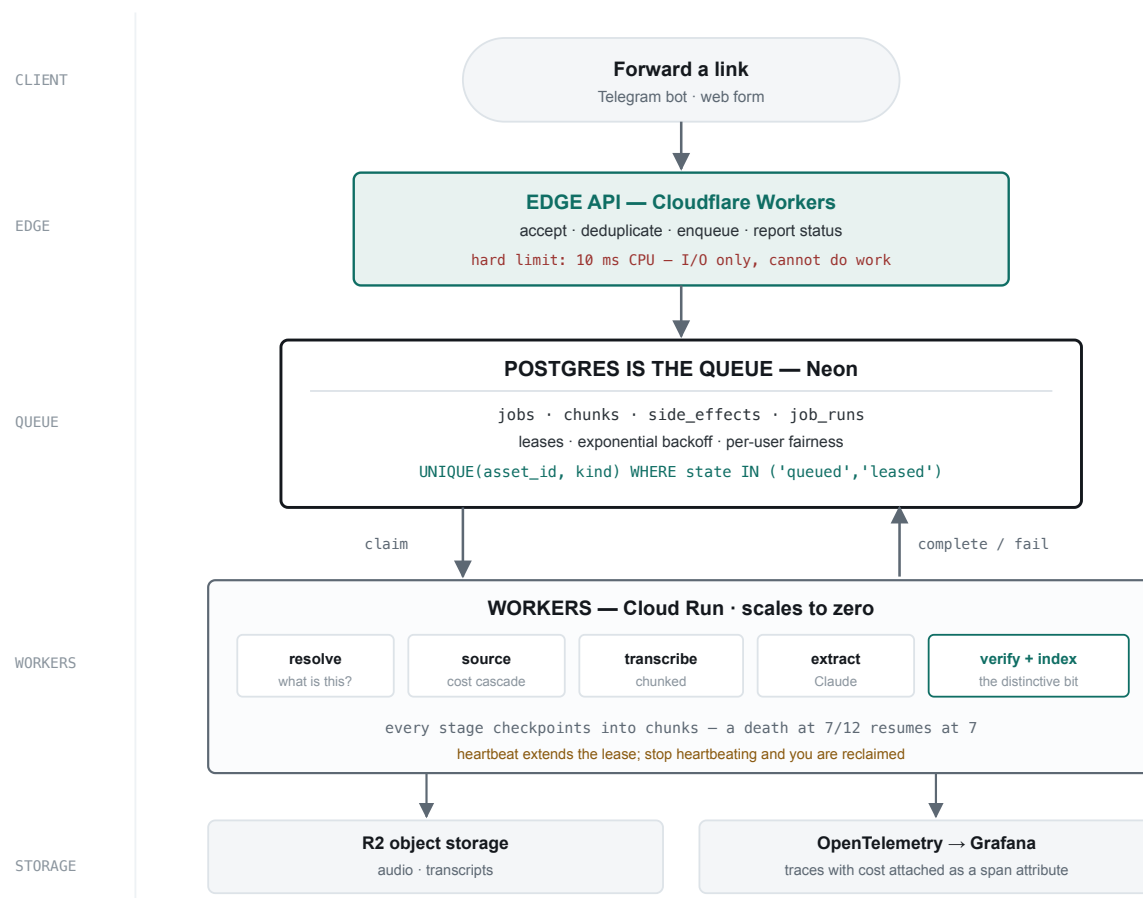


Fig 1 – System architecture. The split at the edge is forced by a 10 ms CPU limit, not chosen for elegance.

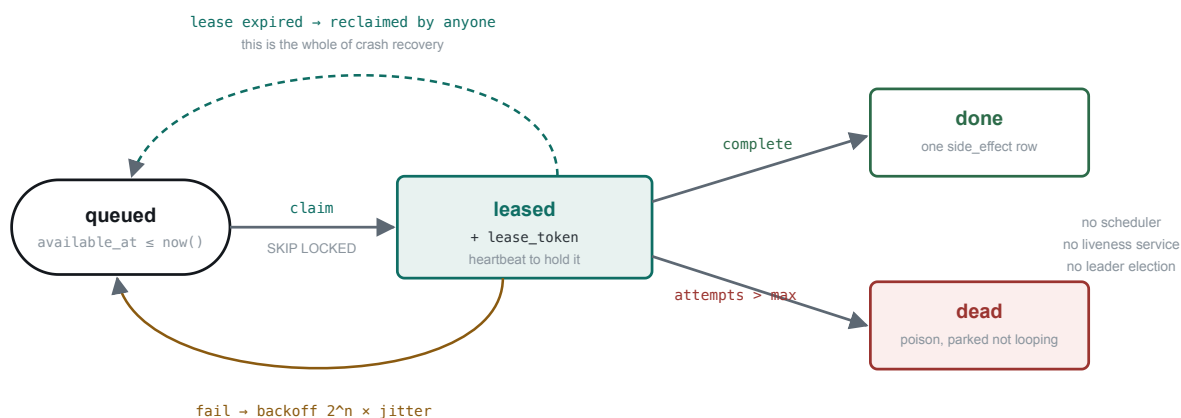
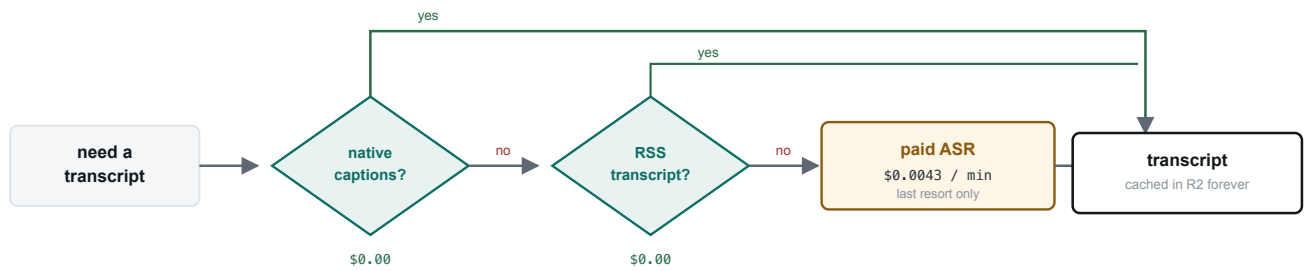


Fig 2 – Job lifecycle. Two timestamps replace an entire scheduler: `available_at` for backoff, `lease_expires_at` for recovery.



Every step records what it cost as an OpenTelemetry span attribute.
The trace therefore answers "why was this item expensive?" without a separate billing system.
content-addressed: 10 users saving one video pay for it once, ever

Fig 3 – The cost cascade. A runtime choice between a free path and a paid one is what makes cost an engineering concern rather than a billing surprise.

06 Every technology, and why

Each row is a decision you should be able to defend in an interview, including its downside.

CHOICE	WHY THIS ONE	THE HONEST DOWNSIDE
TypeScript on Node 22	Your strongest language, so the novelty budget goes on queueing and observability rather than on syntax. Node 22 runs <code>.ts</code> natively, so there is no build step, no bundler, and the repo stays readable.	A polyglot CV would be more impressive to some. Not worth the cost here — learning Go and distributed queues simultaneously would mean learning neither.
Postgres on Neon	Free, hosted, nothing installed locally. Crucially it has partial unique indexes , which lets the in-flight coalescing guarantee live in the database instead of in application code.	Using a relational database as a message broker does not scale to millions of jobs. Correct, and you should say so: at this scale it is the right call, and knowing where it breaks is part of the answer.
Cloudflare Workers	The only genuinely always-on free tier. 100k requests/day and it never cold-starts, so a recruiter clicking the link gets an instant response.	10 ms CPU. Severe, and it dictates the architecture — but the resulting front-door/back-room split is a pattern worth having built.
Cloud Run	Long-running, CPU-heavy work that scales to zero when idle. You already know it from the job pipeline, so it is deliberately the boring part.	Cold starts on the worker path. Irrelevant here — jobs are already asynchronous, so a few seconds of start-up is invisible.
Cloudflare R2	Object storage with no egress fees , which matters when serving cached audio and transcripts.	Another vendor in the stack. Mitigated by touching it through a small interface.
OpenTelemetry → Grafana Cloud	The verified 2026 vendor-neutral standard (CNCF graduated May 2026). Genuinely new to you, and tracing <i>across</i> two runtimes is the interesting part. Attaching money to spans is rare and interviewers find it memorable.	Instrumentation is fiddly and easy to do badly — over-sample and it is noise, under-sample and it is useless.
Claude Haiku	Cheapest capable model, \$1/M input and \$5/M output. Extraction is a structured-output task, not a reasoning marathon.	Weaker than Sonnet on messy transcripts. The eval suite in phase 7–8 is exactly how you find out whether that matters, rather than guessing.

CHOICE	WHY THIS ONE	THE HONEST DOWNSIDE
Deepgram (ASR fallback)	\$0.0043/min batch. Only reached when both free transcript paths fail, so it is a small fraction of items.	The only line item that scales with usage. The cascade exists to keep it rare.
Telegram bot	The lowest-friction way for real people to send you links. No app, no signup, works on everyone's phone.	Not a portfolio-grade UI. Acceptable — real usage matters more than a landing page.

07 What was rejected, and why

Knowing why you did *not* pick something is usually the better interview answer.

REJECTED	REASON
MySQL	It <i>does</i> have <code>FOR UPDATE SKIP LOCKED</code> (8.0.1+), so that was never the differentiator — a common misconception worth correcting. It lacks partial indexes , which would push the dedup guarantee into application code, where guarantees rot.
MongoDB	You already run it, and it is more capable here than it is usually given credit for: <code>findOneAndUpdate</code> is a genuine atomic claim, and partial unique indexes exist via <code>partialFilterExpression</code> . The awkward part is <i>fair</i> queueing — "pick the job whose owner has fewest in flight" becomes an aggregation plus an atomic update, fiddly and racy, versus one SQL statement.
Postgres installed locally	Proposed once and rightly rejected. Nothing needs to go on your Mac; Neon gives real concurrent connections for free. Do not revisit this.
Supabase	The free tier pauses after roughly 7 days of inactivity and recovery is a manual dashboard restore — so a recruiter clicking three weeks later finds a dead demo. Disqualifying for this specific requirement.
Confluent Cloud / managed Kafka	Free credit expires after 30 days, then the organisation is suspended and data becomes inaccessible. Disqualified outright.
SQS / Redis / BullMQ	They would work, and they would also do the learning for you. The mechanisms they hide are the entire point of the project.
Three earlier project ideas	A silent-failure cron monitor, a webhook delivery service, and reconciliation-as-a-service. All rejected by you for the same two reasons: too close to the day job, and not interesting enough to sustain 12 weeks of evenings. That judgement was correct — interest is the resource most likely to run out.

NINETEEN CLAIMS WERE REFUTED — DO NOT USE THEM TO JUSTIFY ANYTHING

An adversarial research pass killed nineteen popular career claims, including: Kubernetes as near-mandatory (the "18.6% of postings" figure), salary premiums for observability, Rust or dbt, "LLM skills appear in only 5% of postings", Go out-earning other languages, specific EU/NL stack-demand percentages, and the widely repeated idea that *recruiters now dismiss AI-assisted portfolios*.

None survived verification. They circulate confidently in career content. If you find yourself justifying a decision with one of them, the decision needs a different reason.

08 The five engineering features

These are the reason the project is not a CRUD app. Each is a direct answer to an interview question.

FEATURE	WHAT IT ACTUALLY DOES	THE QUESTION IT ANSWERS
1 • Cost cascade	Native captions (free) → RSS transcript (free) → paid ASR (last resort). A genuine runtime choice between a free and a paid path, with the cost recorded on the span.	"How do you keep an AI feature from becoming a runaway bill?"
2 • Content-addressed dedup + singleflight	Ten users saving one video produce one asset, one job, ten saves. A second request while a job is in flight <i>attaches as a waiter</i> rather than starting duplicate work — enforced by a partial unique index, not by check-then-insert.	"Two identical requests arrive at the same instant. What happens?"
3 • Resumability	<code>chunks</code> is a table, not a column. A worker that dies at chunk 7 of 12 resumes at 7 and does not re-spend on 1–6. Idempotency keys throughout.	"Your worker dies mid-job. Do you pay twice?"
4 • Fair queueing	Leases, visibility timeouts, backpressure, and per-user fairness. One person pasting 200 links cannot starve everyone else — expressed as <code>ORDER BY in_flight ASC</code> , with no rate limiter or quota table.	"One tenant floods the queue. How do the others still get served?"
5 • Evals	A golden set, schema-validated structured output, prompt-version regression runs, per-field confidence, and user corrections fed back as new eval cases.	"How do you know the extraction is right?" — the one most people cannot answer.

The query you will be asked to walk through

```
WITH in_flight AS (  
  SELECT fairness_user_id AS uid, COUNT(*) AS n FROM jobs  
  WHERE state = 'leased' AND lease_expires_at > now() GROUP BY 1)  
SELECT j.* FROM jobs j LEFT JOIN in_flight f ON f.uid = j.fairness_user_id  
WHERE j.state = 'queued' AND j.available_at <= now()  
ORDER BY COALESCE(f.n, 0) ASC, j.created_at ASC  
FOR UPDATE OF j SKIP LOCKED LIMIT 1;
```

TWO DETAILS THAT WILL BE PROBED

`FOR UPDATE OF j`, not a bare `FOR UPDATE` — Postgres refuses to lock "the nullable side of an outer join", and `in_flight` is LEFT JOINed. A bare `FOR UPDATE` fails at runtime with exactly that message.

`SKIP LOCKED` — without it, ten workers serialise behind a single row lock and throughput collapses to one job at a time regardless of how many workers you run.

09 Phases 1–10, in depth

Each phase lists what you build, what you must understand *before* starting, and the gate you must pass to move on.

THE RULE THAT MAKES THIS WORK

Week 1 uses no AI at all. The worker sleeps and randomly throws. If you add transcription before the queue is provably correct, you will be debugging prompt quality and lease expiry at the same time and you will not know which is which. Earn the queue first.

WEEK 1–2 The queue, and breaking it on purpose

Schema, lease mechanics, and a chaos harness. The worker is *deliberately terrible*: it throws, it hangs past its lease, and it hard-exits mid-job. A separate test spawns real processes and `kill -9`s them while they hold leases.

- `enqueue` with in-flight coalescing via the partial unique index
- `claim` — the fairness query, `SKIP LOCKED`, a fresh lease token per attempt
- `heartbeat`, `complete` (fenced on lease ownership), `fail` (backoff + jitter), `reclaimExpired`
- A harness asserting: every job terminates; exactly one effect per job; zombie completions are rejected; poison dies instead of looping; the flooder does not starve anyone

UNDERSTAND BEFORE YOU START

1. **At-least-once versus exactly-once.** Exactly-once delivery does not exist. What you can have is at-least-once *delivery* with an effectively-once *effect*, which is why `side_effects` uses `job_id` as its primary key. Get this distinction wrong and every later design decision is confused.
2. **Why a lease is not a lock.** A lock is held by a live connection and vanishes when it drops. A lease is a row with an expiry timestamp and survives the worker's death — which is precisely what makes recovery possible without any liveness detection.
3. **Fencing tokens.** Lease expiry alone does not stop a stalled worker from completing, because its clock is not the database's clock. Read Kleppmann on fencing tokens; this is the subtlest idea in week 1 and the most likely to be probed.
4. **What `SKIP LOCKED` actually does** — and what happens without it. Run the harness both ways and watch throughput collapse.
5. **Exponential backoff with jitter.** Without jitter, a batch that failed together retries together forever, in lockstep, hammering whatever is already broken.
6. **Transaction isolation, enough of it.** You need to know why the claim's `SELECT` and `UPDATE` must be in one transaction. You do not need to memorise all four isolation levels.

GATE — you can explain, without notes, why a worker whose lease expired must not be allowed to complete its job.

OUTCOME

A queue you trust because you broke it. This code is already written and typechecks; it has not yet been run against a live database.

WEEK 3–4

The real pipeline

Replace the fake worker with real stages: resolve the URL, run the cost cascade, transcribe in chunks with checkpoints, extract structured data with Claude.

- Provider abstraction so the ASR vendor is swappable — you will want to compare at least two
- Chunked transcription writing each chunk into `chunks` as it completes
- Structured extraction with a schema, validated on the way out, not trusted

UNDERSTAND BEFORE YOU START

1. **Streaming versus buffering large files.** A two-hour podcast will not fit in memory on a small Cloud Run instance. Learn Node streams properly before this phase, not during it.
2. **Idempotency keys.** Every external call that costs money needs one, or a retry becomes a double charge.
3. **Structured output from an LLM.** Schema-constrained generation, and what to do when the model returns something almost-valid. Assume the JSON will be malformed sometimes and plan the recovery path.
4. **Timeouts and cancellation.** `AbortSignal` throughout. An un-cancellable request is a lease you cannot honour.
5. **The difference between a retryable and a terminal error.** Retrying a 400 forever is how a queue quietly stops making progress.

GATE — one real YouTube link goes in and correct structured data comes out, and killing the worker halfway does not re-spend on completed chunks.

OUTCOME

End-to-end on one real link, resumable.

WEEK 5

Observability, with money attached

OpenTelemetry across the edge, the queue and every worker stage, exported to Grafana Cloud. Cost recorded as a span attribute.

UNDERSTAND BEFORE YOU START

1. **Traces, spans, and context propagation.** The hard part is carrying trace context *across a runtime boundary* — from a Cloudflare Worker, through a database row, into a Cloud Run process. That is not a tutorial case and it is the thing worth having done.
2. **Spans versus metrics versus logs**, and when each is the right instrument. Most people reach for logs for everything.
3. **Sampling.** Trace everything and you will exceed the free tier and learn nothing. Head versus tail sampling, and why you always keep the errors.
4. **Cardinality.** Putting a user ID in a metric label is how you accidentally create a million time series and get throttled.

GATE — you can open one trace and say where the time went and where the money went, without consulting the code.

OUTCOME

A trace that shows why one item cost more than another. Rare, and memorable to an interviewer.

WEEK 6 Real users, real contention

Telegram bot, dedup and singleflight under genuine concurrency, fairness verified under load. Give the link to friends.

UNDERSTAND BEFORE YOU START

1. **Webhook security.** Verify Telegram's signature. An unauthenticated webhook is an open door to your queue.
2. **Backpressure.** What the system should do when it is already saturated — the correct answer is rarely "accept it anyway".
3. **The thundering herd.** Ten simultaneous saves of the same trending video is the singleflight path's real test, and the only way to test it is concurrently.
4. **Rate limits as a first-class concern**, both the ones you impose and the ones vendors impose on you.

GATE — a friend pastes 50 links and your own single link still completes promptly.

OUTCOME

Something people actually use, which is a prerequisite for having real incidents to write about.

WEEK 7–8 Evals — proving the AI part is engineered

A golden set of 30–50 items with known-correct extractions. Schema validation, per-field confidence, prompt-version regression runs, and user corrections captured as new eval cases.

UNDERSTAND BEFORE YOU START

1. **How to regression-test a non-deterministic system.** The same input may legitimately produce different output. You need tolerant assertions and aggregate pass rates, not string equality.
2. **Precision and recall, applied to extraction.** Missing a recipe ingredient and inventing one are different failures with different costs.
3. **Grounding and verification.** Cross-check every extracted claim against the source transcript. This is your reconciliation instinct, pointed at a probabilistic system — the distinctive part of the whole project.
4. **Confidence calibration.** A model's self-reported confidence is not probability. Learn what it is and is not good for before relying on it to gate anything.
5. **Prompt versioning.** Treat a prompt as deployed code: version it, and re-run the golden set before shipping a change.

GATE — you change a prompt, run the suite, and can state what improved and what regressed, in numbers.

OUTCOME

A defensible answer to "how do you know it's right?", with data behind it.

WEEK 9–10

Ship it publicly and let it break

Public URL, README, and — the actual deliverable — **written incident reports**. Something will fail: a vendor will change an API, a queue will back up, a cost will surprise you. Write each one up: what broke, how you noticed, what it cost, what you changed.

THIS PHASE IS THE POINT, NOT PADDING

Verified research finding: what makes a project *discussable* is lived incidents narrated with ownership, resolution and a named cost — "battles won with losses" — not a flawless happy path. A self-inflicted outage on a zero-traffic demo does not have that property, which is exactly why week 6 exists to get real users first.

UNDERSTAND BEFORE YOU START

1. **How to write an incident report** that is neither defensive nor self-flagellating. Timeline, impact, root cause, fix, prevention.
2. **Alerting that is worth having.** An alert you routinely ignore is worse than none, because it teaches you to ignore alerts.
3. **Secret hygiene and a public repo.** Scan the history before you publish, not after.

GATE — three incidents written up. Each one is an interview answer you did not have before.

OUTCOME

The interview material. This is what the previous eight weeks were for.

10 Cost estimate

Zero when idle. Under five dollars a month with real use. The architecture is what makes that true.

Fixed infrastructure — free, verified

SERVICE	FREE ALLOWANCE	BEHAVIOUR WHEN IDLE
Neon Postgres	0.5 GB storage 100 CU-hours/month	Scales to zero after 5 min and cannot be disabled on free. Resumes automatically in under 500 ms . Critically, there is <i>no</i> manual restore — unlike Supabase, a link clicked weeks later still works.
Cloudflare Workers	100,000 req/day	Never sleeps. No cold start. This is why it is the front door.
Cloud Run	2M req/month 180k vCPU-s/month	Scales to zero. You pay nothing between jobs.
Cloudflare R2	10 GB storage no egress fees	Static cost only. Transcripts are small; audio is deleted after transcription.
Grafana Cloud	free tier	Adequate with sampling. <i>Verify current limits when you sign up</i> — free-tier terms move.

Variable cost — the only real money

ITEM	UNIT PRICE	WORKING
Claude Haiku extraction	\$1 / M input \$5 / M output	A 30-minute transcript is roughly 7,000 input tokens; structured output roughly 800. About \$0.011 per item .
Deepgram Nova-3 ASR fallback only	\$0.0043 / min (batch)	A 20-minute item costs \$0.086 — but only when both free paths fail. Streaming is \$0.0077/min; you want batch.

Worked monthly estimate — 100 items, 20% needing paid ASR

80 items · free transcript path	\$0.00
20 items · 20 min average · \$0.0043/min	\$1.72
100 items · Claude Haiku extraction · \$0.011	\$1.10
infrastructure (all free tiers)	\$0.00
total	\$2.82 / month

WHY THE CASCADE IS THE COST STORY

Without it, all 100 items would hit paid ASR: $100 \times 20 \text{ min} \times \$0.0043 = \text{\$8.60}$, plus extraction — roughly **3.5x** the bill. And with content-addressed dedup, ten people saving the same video pay for it *once, ever*. That is a sentence with a number in it, which is what makes it an interview answer rather than an opinion.

SET A HARD CAP BEFORE THE FIRST PAID CALL

You have already been bitten by this: the job pipeline's Anthropic account hit a zero balance mid-run, and because errors were caught per-stage it failed *silently* — jobs simply stopped being scored. Build the spend ledger and the daily cap in week 3, before the first Deepgram call, not after the first surprise.

11 Timeline and effort

10 weeks of building at your real availability, and the honest range is 10–14.

WEEKS	PHASE	NEW TO YOU	RISK OF SLIPPING
1–2	Queue + chaos harness	high	Low. Code is written already. Risk is understanding, not typing.
3–4	Real pipeline	medium	Medium. Vendor APIs and large-file streaming always take longer than planned.
5	OpenTelemetry	very high	High. Cross-runtime context propagation is the single most likely week to overrun. Budget two.
6	Bot, dedup, fairness under load	medium	Low. Mostly exercising what already exists.
7–8	Evals	high	Medium. Building the golden set is tedious manual work and easy to defer forever.
9–10	Ship, break, write up	low	Depends on users. No traffic means no incidents means no material.

Your availability: 1–2 hours on weekdays plus weekends — call it 12–15 hours a week. At that rate the plan is 10 weeks of building if nothing slips, and **realistically 12–14**. Week 5 is the one to watch.

THE RATIO THAT MAKES THIS THE RIGHT PROJECT

Roughly **40% familiar** — TypeScript, Node, SQL, Cloud Run, the Claude API, retry instincts — and **60% new**: leases, visibility timeouts, backpressure, per-tenant fairness, OpenTelemetry, evaluating non-deterministic output. Much more familiar and you learn nothing; much more new and you stall out in week 3. This ratio is why the earlier proposals were rejected and this one was not.

THE SINGLE BIGGEST RISK IS NOT TECHNICAL

It is abandonment at 60%. A half-built project is worth less than no project, because the interview question becomes "why did you stop?" The mitigation is built into the plan: **weeks 1–2 produce something demonstrable on their own** — a queue with a chaos report is already a legitimate thing to talk about, even if you never finish the product. Every phase after that adds a complete layer rather than a partial one.

And the job search remains the priority. This project exists to serve it, not to replace it. If a week has to give, this one gives.

12 What you will be able to claim

Stated as an interviewer would hear it, not as a feature list.

THE CLAIM	THE EVIDENCE BEHIND IT
"I built a job queue on Postgres with leases, backoff and per-tenant fairness."	One SQL statement you can whiteboard, and a chaos harness that proves the properties hold under <code>kill -9</code> .
"I made cost an engineering concern rather than a billing surprise."	A three-step cascade, cost recorded on spans, and a measured 3.5× difference against the naive path.
"I know the difference between at-least-once delivery and an effectively-once effect."	A <code>side_effects</code> table whose primary key makes double-processing raise rather than pass silently.
"I traced a request across two runtimes and attached money to the spans."	A Grafana trace from a Cloudflare Worker through a Postgres row into a Cloud Run worker.
"I regression-test a non-deterministic system."	A golden set, per-field confidence, and prompt-version runs with before-and-after numbers.
"Here is an outage I caused, how I found it, what it cost, and what I changed."	Three written incident reports from a system with real users.

THE ONE-SENTENCE PITCH, BY AUDIENCE

Recruiter: "It turns the stuff you save and never watch into something you can actually search."

Engineer: "It is a content-addressed job pipeline on Postgres — leases, fair scheduling, resumable chunked work — wrapped around an expensive non-deterministic operation, with an eval suite that tells me when the extraction is wrong."

Hiring manager: "I had never built a real queue — everything I'd shipped was cron-driven. So I built one, broke it deliberately, and wrote up what I learned when it broke for real."

13 Risks and honest warnings

RISK	LIKELIHOOD	WHAT TO DO ABOUT IT
Abandoned at 60%	High	The real risk. Weeks 1–2 are self-contained on purpose, so there is always something finished to point at.
Week 5 (OpenTelemetry) overruns	High	Budget two weeks. Cross-runtime context propagation is genuinely hard and poorly documented for this combination.
No real users, so no incidents	Medium	Ship the Telegram bot to friends in week 6, not week 10. Without traffic, phase 9–10 has nothing to write about.
It drifts into a CRUD app	Medium	Re-read your own objection in section 01 whenever a week feels easy. If a week involves no queue, no cost decision and no verification, it is the wrong week.
A free tier changes underneath you	Medium	Already bitten Supabase and Confluent. Keep vendors behind small interfaces so swapping one is a day, not a rewrite.
Transcription bill surprises you	Low	Hard daily cap and a spend ledger in week 3, before the first paid call.
The job search stalls while you build	High	The one that actually matters. Applications sent is the metric; this project is support, not substitute. If a week must give, this one gives.

READ THIS BEFORE WEEK 1

You already have the week 1–2 code written and typechecking. That is *not* the same as understanding it, and the entire value of this project is that you can defend it under questioning. Either re-derive `queue.ts` yourself using the existing version as a reference, or sit with it until you can explain the fencing token, the partial index and `FOR UPDATE OF j` without looking.

"I have a queue I did not write" is the one outcome that fails the brief completely.

Immediate next step

One thing blocks everything: a **Neon connection string** in `/Users/Me/resurface/.env`. Create a free project at neon.tech, copy the pooled string — the one containing `-pooler` — and then:

```
npm run migrate      # apply 001_init.sql
npm run chaos        # 8 workers, deliberate failures, 5 assertions
npm run kill-test    # real processes, kill -9 mid-job
```

Expect the first run to fail something. That is the week working as designed.